รายงานโครงงาน

ระบบ POS ร้านกาแฟ

Pad-Krapow-cafe-system

ระบบรับออเดอร์ คิดเงิน ตัดสต็อก และรายงานยอดขาย สำหรับร้านกาแฟขนาดเล็ก

จัดทำโดย กลุ่มที่ 1 — Pad-Krapow-cafe

67160031 นายธีรเดช อยู่คง

67160030 นายดรัณภพ ทวีพูล

67160039 นายภัคพล ใบเนียม

67160047 นายอภิชาติ ชนะชัย

67160018 นายภูมิพัฒน์ ขันอาสา

ตุลาคม 2569

สารบัญ

บทที่ 1 บทนำ....4
1.1 ความเป็นมาและปัญหา....4
1.2 วัตถุประสงค์....4
1.3 ขอบเขตของระบบ....4
1.4 ผู้ใช้งานระบบ....4
1.5 แนวทางการพัฒนา....5
บทที่ 2 การศึกษาความเป็นไปได้....6
2.1 กระบวนการทำงานเดิมและใหม่ (BPMN)....6
บทที่ 3 การวิเคราะห์ความต้องการ....8
3.1 Functional Requirements....8
3.2 Non-functional Requirements....8
3.3 Use Case Diagram....8
3.4 User Stories....9
3.5 การปรับปรุง Requirement ระหว่างโครงงาน....9
บทที่ 4 การออกแบบระบบ....11
4.1 สถาปัตยกรรมระบบ....11
4.2 Logical View และ Class Diagram....12
4.3 การออกแบบฐานข้อมูล....13
4.4 Process View — ลำดับการสร้างออเดอร์....15
4.5 การออกแบบส่วนติดต่อผู้ใช้....16
บทที่ 5 การพัฒนาระบบ....17
5.1 เครื่องมือและเทคโนโลยี....17
5.2 โครงสร้างซอร์สโค้ด....17
5.3 กลไกสำคัญ....17
5.4 REST API....18
5.5 หน้าจอระบบ....18
บทที่ 6 การทดสอบระบบ....23
6.1 การทดสอบอัตโนมัติ....23
6.2 การทดสอบ End-to-End ผ่านหน้าจอ....23
6.3 การทดสอบประสิทธิภาพ (NFR-01)....23
6.4 Usability Testing....24
บทที่ 7 การตรวจสอบคุณภาพ....25
7.1 สิ่งที่แก้จากโค้ดเดิม....25
7.2 ประเด็นที่ยังควรปรับปรุง....25
บทที่ 8 สรุปและข้อเสนอแนะ....26
8.1 สรุปผล....26
8.2 ข้อจำกัด....26

8.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ....26
ภาคผนวก....27
ก. วิธีติดตั้งและรันระบบ....27
ข. บัญชีสำหรับทดลอง....27
ค. การใช้ AI ในโครงงาน....27
ง. เอกสารอ้างอิงใน Repository....27

บทที่ 1 บทนำ

1.1 ความเป็นมาและปัญหา

ร้านกาแฟกรณีศึกษาเป็นร้านขนาดเล็กที่ยังใช้สมุดจดออเดอร์และเครื่องคิดเลขในการคิดเงิน จากการสัมภาษณ์เจ้าของร้าน (wk03) พบปัญหาหลัก 3 ข้อ ได้แก่

1. **รับออเดอร์และคิดเงินผิดบ่อย** โดยเฉพาะช่วงเร่งด่วนตอนเช้าที่ลูกค้าเยอะ ทำให้ลูกค้ารอนาน
2. **สต็อกหมดโดยไม่รู้ตัว** เช่น นมหมดกะทันหันจนต้องปฏิเสธลูกค้า หรือสั่งของซ้ำเพราะจำไม่ได้ว่าเหลือเท่าไร
3. **ไม่มีข้อมูลยอดขาย** นับเงินสดปลายวันแล้วจดลงสมุด ไม่รู้ว่าเมนูไหนขายดี

1.2 วัตถุประสงค์

- พัฒนาระบบ POS แบบเว็บที่ใช้กับแท็บเล็ตและคอมพิวเตอร์ที่ร้านมีอยู่ ไม่ต้องลงทุนอุปกรณ์เพิ่ม
- ลดความผิดพลาดในการรับออเดอร์และคิดเงิน โดยให้ระบบคำนวณยอดเงินและตัดสต็อกอัตโนมัติ
- แจ้งเตือนเมื่อสต็อกใกล้หมด และป้องกันการขายเกินสต็อก
- ให้เจ้าของร้านดูยอดขายรายวัน รายเดือน และเมนูขายดีได้ทันที

1.3 ขอบเขตของระบบ

ตารางที่ 1 ขอบเขตที่ครอบคลุมและไม่ครอบคลุม

ระบบทำ	ระบบไม่ทำ (เวอร์ชันนี้)
เข้าสู่ระบบ แยกบทบาทบาริสต้า / เจ้าของร้าน	ระบบบัญชีและภาษีเต็มรูปแบบ
รับออเดอร์ คำนวณยอด เลือกวิธีชำระ (เงินสด/โอน/QR พร้อมเพย์)	สั่งซื้อวัตถุดิบจากซัพพลายเออร์อัตโนมัติ
ตัดสต็อกอัตโนมัติ + ประวัติสต็อก + แจ้งเตือนใกล้หมด	ระบบสมาชิก / สะสมแต้ม / ส่วนลด
ใบเสร็จ (แสดง พิมพ์ เปิดย้อนหลัง)	ยกเลิก / คืนเงินออเดอร์
จัดการเมนูและเติมสต็อก	เชื่อมต่อเครื่องรับบัตร / QR อัตโนมัติ
รายงานยอดขายรายวัน / ช่วงวันที่ / รายเดือน + Top 5	การทำงานแบบ offline
รองรับหลายสาขาในฐานข้อมูลเดียว	สูตรวัตถุดิบต่อเมนู

1.4 ผู้ใช้งานระบบ

บทบาท	หน้าที่ในระบบ
ลูกค้า	ไม่ใช้ระบบโดยตรง แจ้งเมนู ชำระเงิน รับใบเสร็จ
บาริสต้า	รับออเดอร์ ดูใบเสร็จ ดูและเติมสต็อก ดูรายงาน (เฉพาะสาขาตัวเอง)

บทบาท	หน้าที่ในระบบ
เจ้าของร้าน	ทำได้ทุกอย่างในทุกสาขา และเพิ่ม/แก้ไข/ซ่อนเมนู

1.5 แนวทางการพัฒนา

ทีมเลือกใช้ **Waterfall Model** (wk01) เพราะระบบร้านกาแฟมีขั้นตอนชัดเจน กำหนดความต้องการได้ก่อนพัฒนา และแบ่งงานเป็นลำดับ วิเคราะห์ → ออกแบบ → พัฒนา → ทดสอบ → ติดตั้ง ได้ง่าย อย่างไรก็ตามในช่วง Coding Sprint มีการย้อนกลับไปปรับ requirement และ diagram ให้ตรงกับสิ่งที่พบระหว่างพัฒนา ซึ่งบันทึกไว้ใน Requirement Revision Log

บทที่ 2 การศึกษาความเป็นไปได้

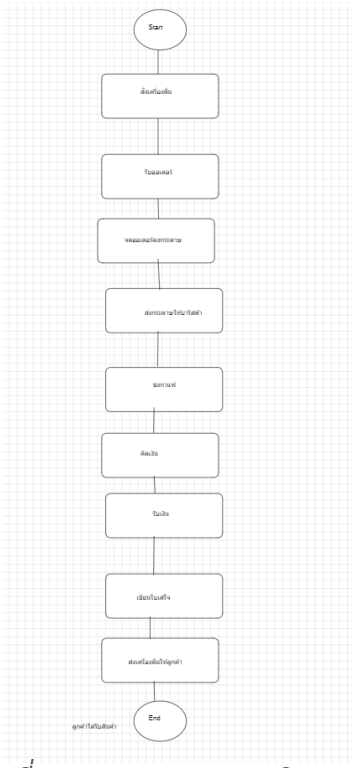
ประเมินความเป็นไปได้ 4 ด้านตาม Feasibility Report (wk02)

ตารางที่ 2 สรุปผลการศึกษาความเป็นไปได้

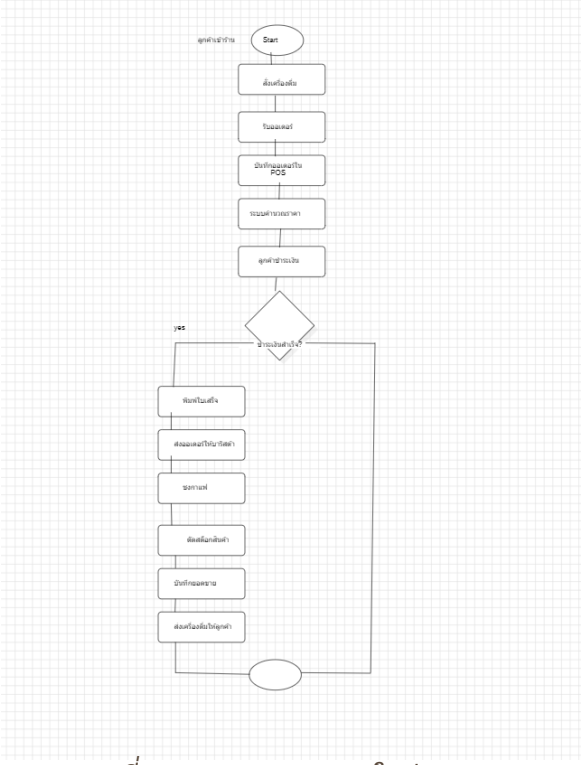
ด้าน	สาระสำคัญ	ผลประเมิน
Technical	ใช้ Node.js, Express, MySQL, HTML/CSS/JS ซึ่งเป็นเทคโนโลยีที่ทีมมีพื้นฐานและมีเอกสารจำนวนมาก	เป็นไปได้สูง
Economic	ต้นทุนเริ่มต้นประมาณ 45,000 บาท ค่าใช้จ่ายประจำประมาณ 1,000 บาท/เดือน คืนทุนประมาณ 6–8 เดือน	คุ้มค่า
Operational	เจ้าของร้านสนับสนุน พนักงานใช้สมาร์ทโฟนคล่อง อบรมประมาณ 2 ชั่วโมง	ใช้งานได้จริง
Schedule	แผน 16 สัปดาห์ วิเคราะห์ → ออกแบบ → พัฒนา → ทดสอบ → ส่งมอบ	ทันกำหนด

หมายเหตุ: ประมาณการต้นทุนใน wk02 รวมค่าซื้อคอมพิวเตอร์และเครื่องพิมพ์ใบเสร็จ แต่ข้อจำกัดใน wk03 ระบุว่าร้านมีแท็บเล็ตและคอมพิวเตอร์อยู่แล้ว ระบบที่พัฒนาจึงใช้ได้กับอุปกรณ์เดิม (พิมพ์ใบเสร็จผ่านเบราว์เซอร์) ต้นทุนจริงจึงอาจต่ำกว่าที่ประมาณไว้

2.1 กระบวนการทำงานเดิมและใหม่ (BPMN)



ภาพที่ 1 BPMN กระบวนการเดิม (As-Is)



ภาพที่ 2 BPMN กระบวนการใหม่ (To-Be)

ตารางที่ 3 เปรียบเทียบ As-Is และ To-Be

As-Is	To-Be (ระบบที่พัฒนา)
จดออเดอร์ด้วยกระดาษ	แตะเมนูบนหน้าจอ POS
คิดเงินด้วยเครื่องคิดเลข	ระบบคำนวณยอดรวมให้ และ server คำนวณซ้ำจากราคาในฐานข้อมูล
เขียนใบเสร็จ	แสดงและพิมพ์ใบเสร็จจากระบบ
ไม่ได้ตัดสต็อก	ตัดสต็อกอัตโนมัติพร้อมแจ้งเตือนใกล้หมด
ไม่มีรายงาน	รายงานยอดขายรายวัน/รายเดือนแบบทันที

บทที่ 3 การวิเคราะห์ความต้องการ

รวบรวมความต้องการจากการสัมภาษณ์เจ้าของร้าน (wk03) แปลงเป็น Use Case และ User Story (wk04) และสรุปเป็น SRS (wk10)

3.1 Functional Requirements

ตารางที่ 4 Functional Requirements

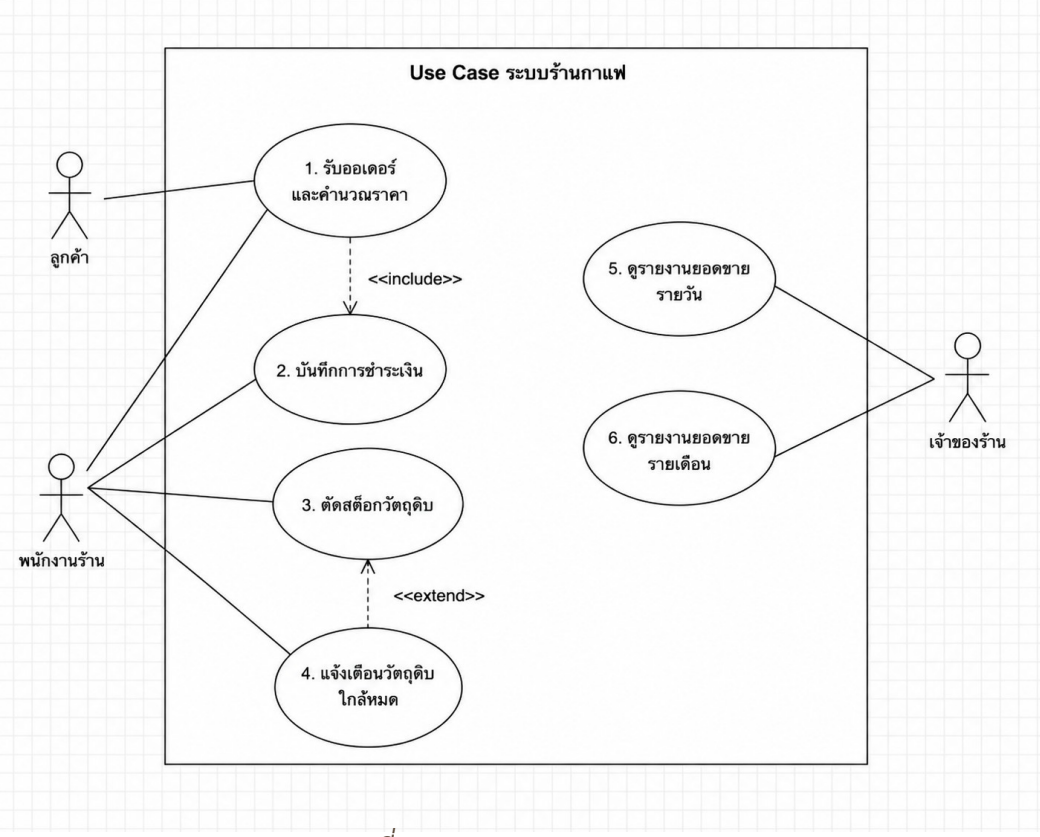
รหัส	Requirement	User Story
FR-01	บันทึกรายการสั่งซื้อ ระบุเมนูและจำนวน	US-01, US-03
FR-02	คำนวณยอดเงินรวมอัตโนมัติ	US-02
FR-03	บันทึกวิธีชำระเงิน (เงินสด/โอน/QR พร้อมเพย์)	US-04
FR-04	ตัดสต็อกอัตโนมัติเมื่อยืนยันขาย	US-05
FR-05	แจ้งเตือนเมื่อสต็อกต่ำกว่าจำนวนขั้นต่ำ	US-06
FR-06	สรุปยอดขายรายวันแยกตามเมนู	US-07
FR-07	สรุปยอดขายรายเดือน + เมนูขายดี 5 อันดับ	US-08
FR-08	แสดงสต็อกคงเหลือแบบเรียลไทม์	US-05, US-06
FR-09	ใช้งานผ่านแท็บเล็ตโดยไม่ติดตั้งเพิ่ม	—
FR-10*	ห้ามขายเกินสต็อก และห้ามสั่งเมนูที่หมด	US-05
FR-11*	แสดงและพิมพ์ใบเสร็จ	UC-01
FR-12*	เจ้าของร้านจัดการเมนู	—
FR-13*	เข้าสู่ระบบและจำกัดสิทธิ์ตามบทบาท/สาขา	—

* เพิ่มระหว่าง Coding Sprint และบันทึกใน Requirement Revision Log

3.2 Non-functional Requirements

รหัส	Requirement
NFR-01	บันทึกออเดอร์และคำนวณยอดเสร็จภายใน 2 วินาที
NFR-02	พนักงานใหม่บันทึกออเดอร์แรกได้ภายใน 5 นาทีหลังอบรม
NFR-03	เข้ารหัสรหัสผ่านด้วย bcrypt
NFR-04	รองรับ ≥ 3 อุปกรณ์พร้อมกัน ข้อมูลตรงกันภายใน 3 วินาที
NFR-05*	ป้องกัน SQL injection, XSS, IDOR และไม่เชื่อราคา/รหัสพนักงานจาก client

3.3 Use Case Diagram



ภาพที่ 3 Use Case Diagram (wk04)

3.4 User Stories

ID	As a	I want	So that
US-01	บาริสต้า	บันทึกรายการเมนูและจำนวน	สร้างออเดอร์ได้ถูกต้อง
US-02	บาริสต้า	ให้ระบบคำนวณยอดรวมอัตโนมัติ	แจ้งราคาลูกค้าได้ถูกต้อง
US-03	บาริสต้า	เพิ่มหรือแก้ไขจำนวนในออเดอร์	ออเดอร์ตรงกับที่ลูกค้าต้องการ
US-04	บาริสต้า	บันทึกช่องทางชำระเงิน	เก็บข้อมูลการชำระของแต่ละออเดอร์
US-05	บาริสต้า	ให้ระบบตัดสต็อกหลังยืนยันขาย	ข้อมูลสต็อกถูกต้อง
US-06	บาริสต้า	ได้รับแจ้งเตือนเมื่อของใกล้หมด	เตรียมสั่งของได้ทัน
US-07	เจ้าของร้าน	ดูยอดขายรายวันแยกตามเมนู	ตรวจผลการขายแต่ละวัน
US-08	เจ้าของร้าน	ดูยอดขายรายเดือน	วิเคราะห์ยอดขายและเมนูขายดี

3.5 การปรับปรุง Requirement ระหว่างโครงงาน

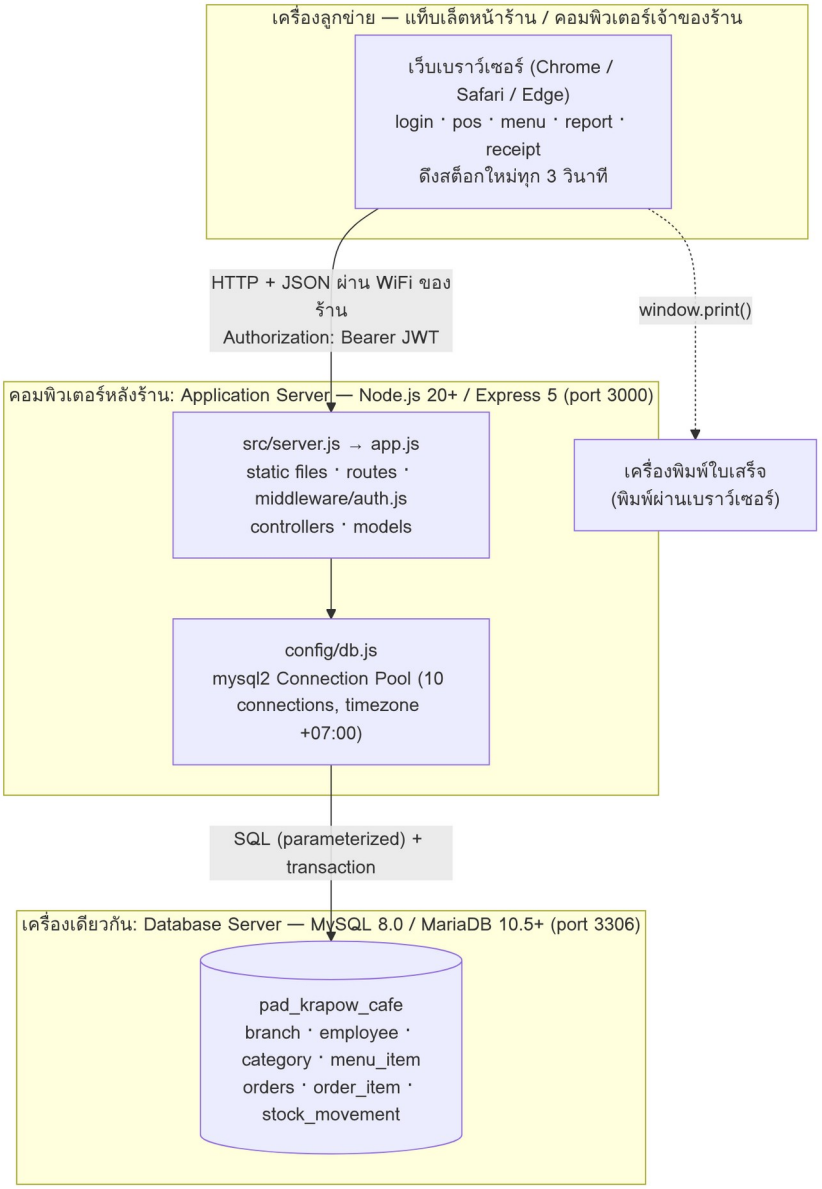
ครั้งที่	การเปลี่ยนแปลง	เหตุผล
1	จาก 1 สาขา → รองรับหลายสาขาด้วย branch_id	Wireframe wk10 กำหนดตัวเลือกสาขาทุกหน้าจอ
2	"สต็อกวัตถุดิบ" → นับเป็นจำนวนที่ขายได้ต่อเมนู	ตาม ER Diagram wk07

ครั้งที่	การเปลี่ยนแปลง	เหตุผล
3	ขั้นต่ำแจ้งเตือนค่าคงที่ 10 → ตั้งได้ทีละเมนู	ขนม/เบเกอรี่ทำทีละน้อย
4–7	เพิ่ม FR-10 ถึง FR-13	พบช่องว่างระหว่างพัฒนาและทดสอบ
8	รองรับ offline → แจ้งเตือนเมื่อเชื่อมต่อไม่ได้	ลดความเสี่ยงขายเกินสต็อก

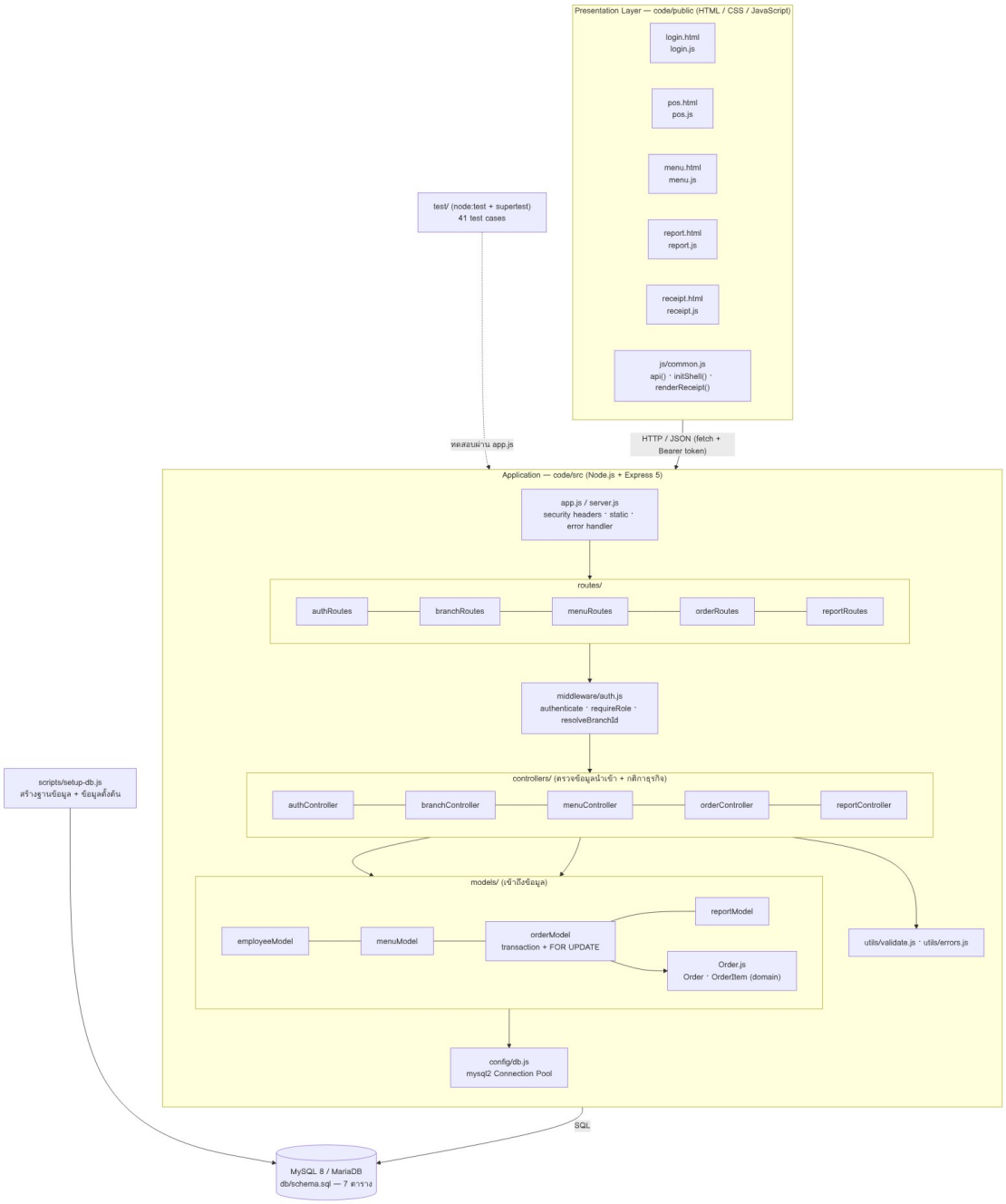
บทที่ 4 การออกแบบระบบ

4.1 สถาปัตยกรรมระบบ

ระบบเป็น **Monolithic แบบแบ่งชั้น (Layered)** 3-Tier: Presentation (HTML/CSS/JS) → Application (Node.js + Express) → Data (MySQL) เหตุผลคือทีมเล็ก มีเวลา 1 ภาคการศึกษา และทุกฟังก์ชันใช้ข้อมูลร่วมกัน ทำให้ใช้ database transaction ครอบการบันทึกออเดอร์และตัดสต็อกได้โดยตรง (รายละเอียด 4+1 View ใน wk11)



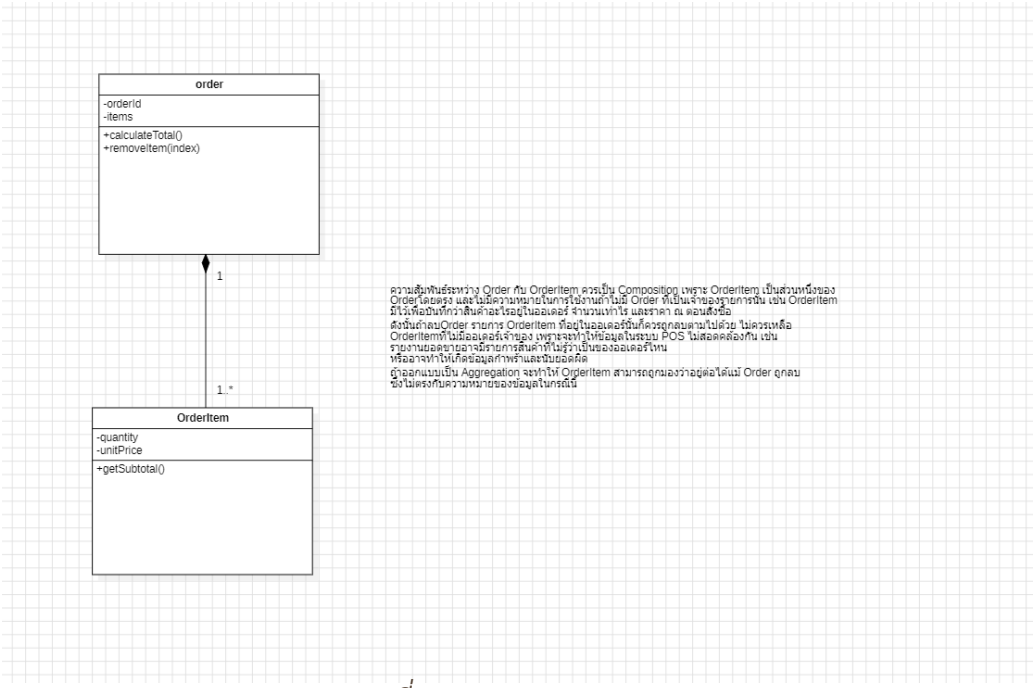
ภาพที่ 4 Physical View



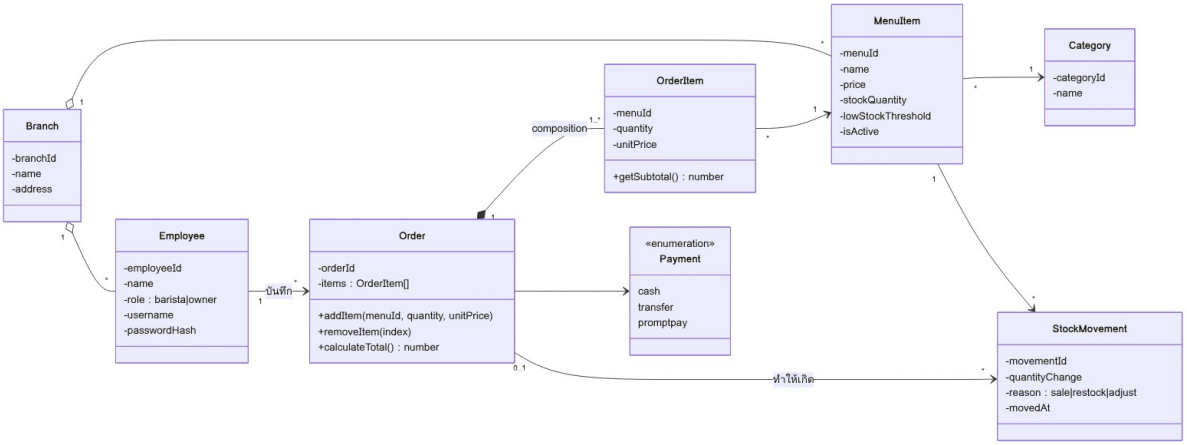
ภาพที่ 5 Development View (โครงสร้างโค้ดจริง)

4.2 Logical View และ Class Diagram

Order กับ OrderItem เป็น **Composition** เพราะ OrderItem ไม่มีความหมายถ้าไม่มี Order (wk06) ในโค้ดคือไฟล์ `src/models/Order.js`

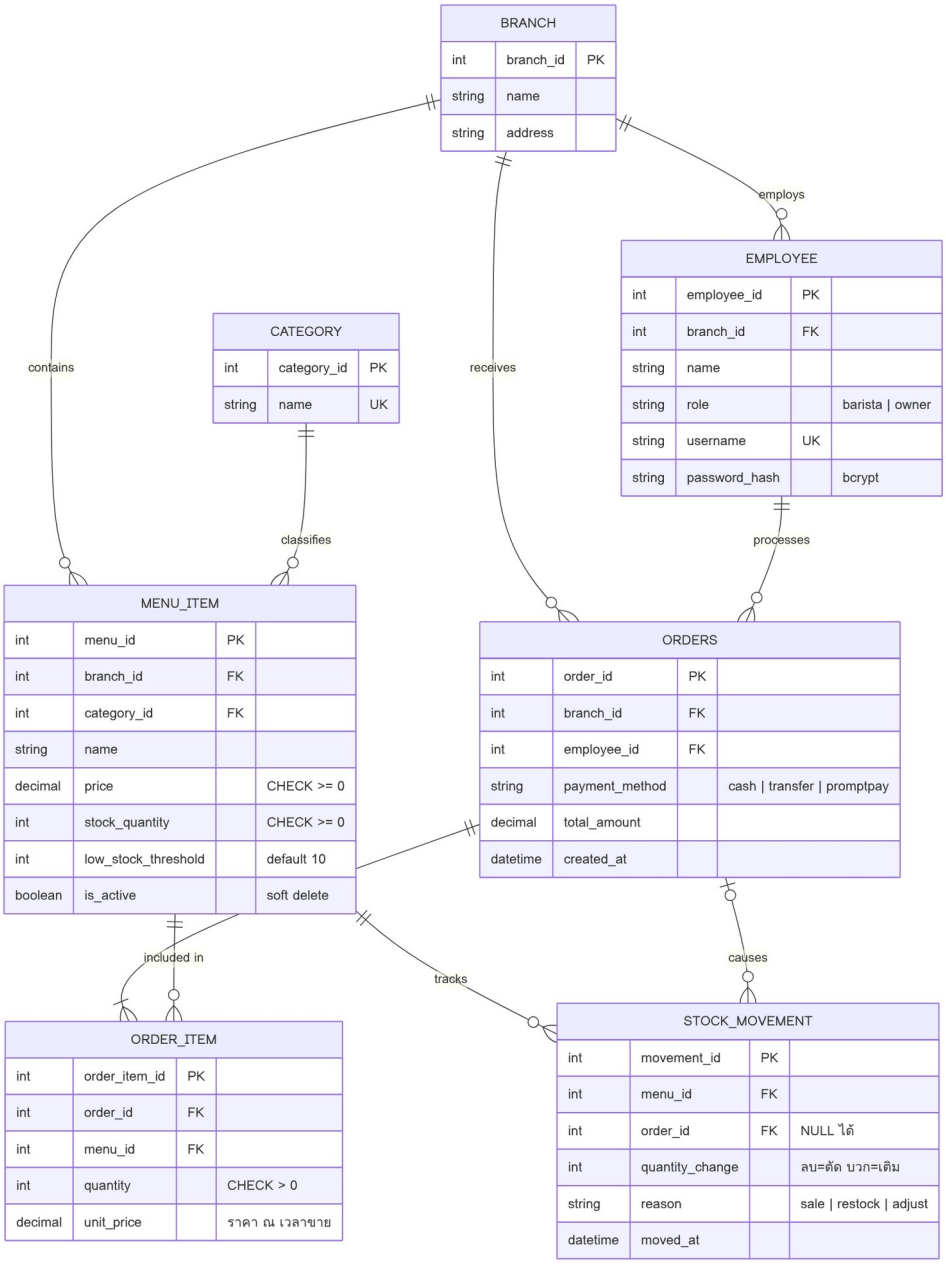


ภาพที่ 6 Class Diagram (wk06)



ภาพที่ 7 Logical View ของระบบที่พัฒนา

4.3 การออกแบบฐานข้อมูล



ภาพที่ 8 ER Diagram (ฉบับปรับปรุง)

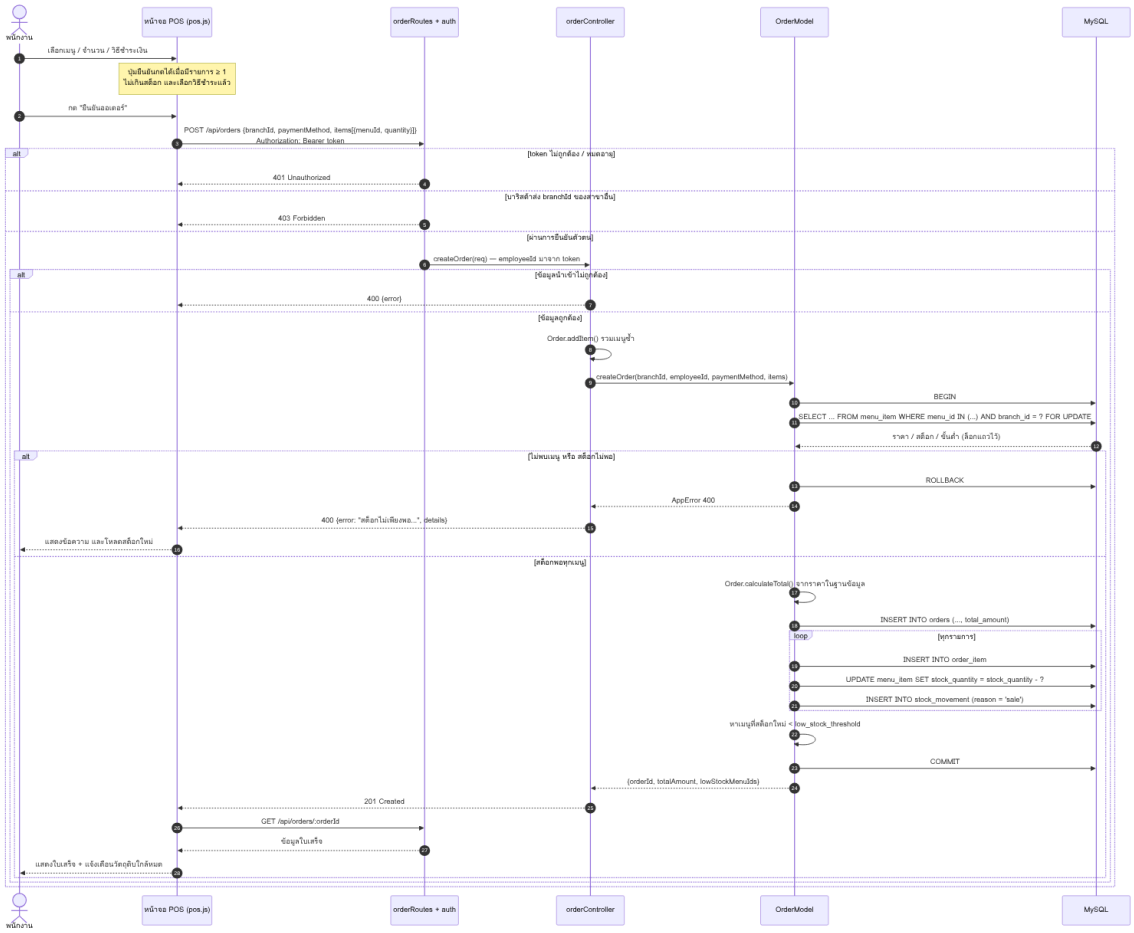
ตารางที่ 5 ตารางในฐานข้อมูล

ตาราง	เก็บข้อมูล	กติกาสำคัญ
branch	สาขา	—
category	หมวดหมู่เมนู	ชื่อห้ามซ้ำ
employee	พนักงาน + บัญชีผู้ใช้	role ∈ {barista, owner}, รหัสผ่านเป็น bcrypt
menu_item	เมนูของแต่ละสาขา	ราคา ≥ 0, สต็อก ≥ 0, low_stock_threshold, is_active
orders	หัวออเดอร์	payment_method ∈ {cash, transfer, promptpay}, total_amount

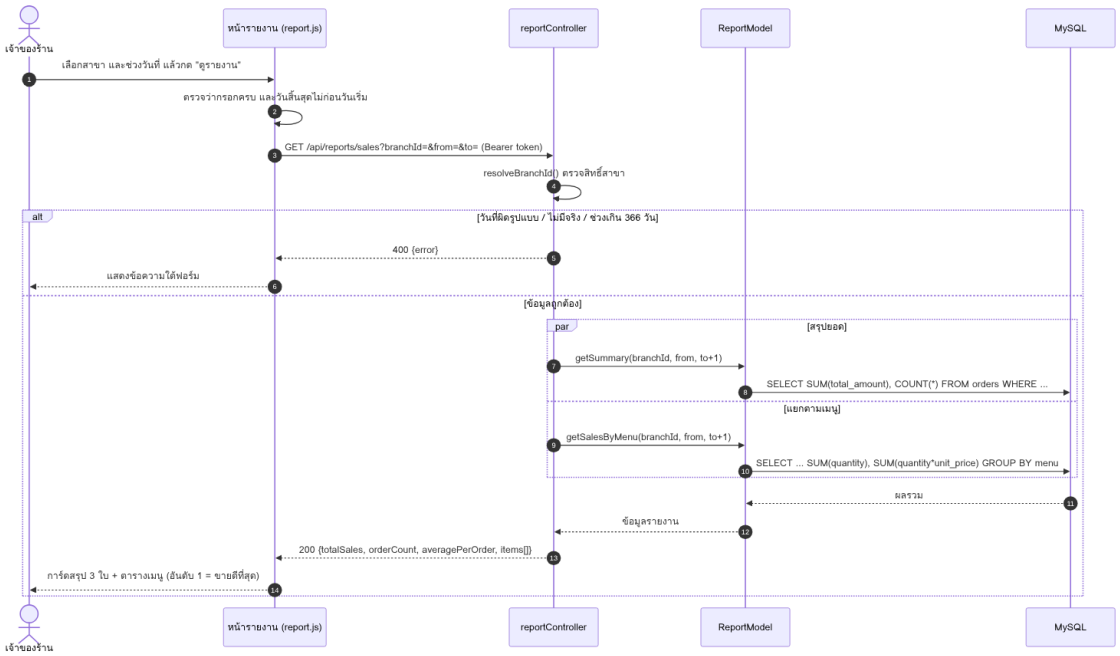
ตาราง	เก็บข้อมูล	กติกาสำคัญ
order_item	รายการในออเดอร์	quantity > 0, unit_price = ราคา ณ เวลาขาย
stock_movement	ประวัติสต็อก	reason ∈ {sale, restock, adjust}

4.4 Process View — ลำดับการสร้างออเดอร์

ปรับจาก Sequence Diagram wk09 ให้ตรงกับโค้ด: ตรวจสิทธิ์ด้วย token → ตรวจข้อมูลนำเข้า → รวมเมนูซ้ำ → เปิด transaction → ล็อกแถวเมนูด้วย `SELECT ... FOR UPDATE` → ถ้าสต็อกไม่พอ rollback ตอบ 400 → ถ้าพอ บันทึกออเดอร์ ตัดสต็อก บันทึกประวัติ แล้ว commit ตอบ 201 พร้อมรายการเมนูที่ใกล้หมด



ภาพที่ 9 Sequence Diagram: POST /api/orders



ภาพที่ 10 Sequence Diagram: GET /api/reports/sales

4.5 การออกแบบส่วนติดต่อผู้ใช้

ออกแบบจาก Wireframe และ Input/Output Design (wk10) โดยยึดหลัก Consistency, Feedback, Error Prevention, Visibility และ Minimalist Design ตัวอย่างการนำไปใช้:

- **Error Prevention:** ปุ่ม "ยืนยันออเดอร์" กดไม่ได้จนกว่าจะมีรายการและเลือกวิธีชำระ พร้อมข้อความบอกว่ายังขาดอะไร ปุ่ม + ถูกปิดเมื่อถึงจำนวนสต็อก
- **Feedback:** toast แจ้งผลสำเร็จ/ผิดพลาด แจ้งเตือนเมนูใกล้หมดทันทีหลังขาย
- **Visibility:** การ์ดเมนูแสดงสต็อก ป้าย "⚠️ ใกล้หมด" และ "หมด" เมนูด้านซ้ายเข้าถึงทุกหน้าใน 1 คลิก
- **Consistency:** แถบด้านบน (สาขา, ชื่อพนักงาน) และเมนูด้านซ้ายเหมือนกันทุกหน้า

บทที่ 5 การพัฒนาระบบ

5.1 เครื่องมือและเทคโนโลยี

ส่วน	เทคโนโลยี
Backend	Node.js 20+, Express 5, mysql2 (Connection Pool), bcryptjs, jsonwebtoken, dotenv
Frontend	HTML, CSS, JavaScript ล้วน (ไม่ใช้ framework) ออกแบบ responsive สำหรับแท็บเล็ต
Database	MySQL 8.0 / MariaDB 10.x
Testing	node:test, supertest, Playwright (E2E)
Diagram	StarUML และ Mermaid (ไฟล์ต้นฉบับอยู่ใน docs/diagrams/)

5.2 โครงสร้างซอร์สโค้ด

โฟลเดอร์ / ไฟล์	หน้าที่
code/src/server.js, app.js	เปิด server, security headers, routes, error handler กลาง
code/src/config/db.js	Connection Pool ตั้งเวลาไทย และให้ DECIMAL เป็นตัวเลข
code/src/middleware/auth.js	ตรวจ JWT, บทบาท และสิทธิ์สาขา
code/src/routes/ (5 ไฟล์)	auth, branch, menu, order, report
code/src/controllers/ (5 ไฟล์)	ตรวจข้อมูลนำเข้า + กติกาธุรกิจ
code/src/models/	Order.js (domain), employeeModel, menuModel, orderModel, reportModel
code/public/	หน้าจอ login, pos, menu, report, receipt
code/db/schema.sql, scripts/setup-db.js	สร้างฐานข้อมูลและข้อมูลตั้งต้น
code/test/	ชุดทดสอบอัตโนมัติ 41 กรณี

5.3 กลไกสำคัญ

การสร้างออเดอร์และตัดสต็อก

`OrderModel.createOrder()` ทำทุกขั้นตอนใน transaction เดียว ล็อกแถวเมนูด้วย `FOR UPDATE` เรียงตาม menu_id เพื่อลดโอกาส deadlock ตรวจสต็อกก่อนตัด ถ้าไม่พอจะ rollback ทั้งหมด ราคาอ่านจากฐานข้อมูลเท่านั้น และคำนวณยอดเป็นหน่วยสตางค์เพื่อเลี่ยงปัญหาทศนิยม จากการทดสอบส่ง 8 ออเดอร์พร้อมกันบนสต็อก 3 ชิ้น ระบบขายได้ 3 และปฏิเสธ 5 สต็อกไม่ติดลบ

การยืนยันตัวตนและสิทธิ์

รหัสผ่านเก็บเป็น bcrypt hash หลัง login ได้ JWT อายุ 12 ชั่วโมง ทุกคำขอต้องแนบ token บาริสต้าเข้าถึงได้เฉพาะสาขาตัวเอง (ส่งสาขาอื่นได้ 403) รหัสพนักงานที่บันทึกในออเดอร์มาจาก token ไม่ใช่จากข้อมูลที่ส่งมา ใส่รหัสผิด 5 ครั้งใน 15 นาทีจะถูกพักชั่วคราว

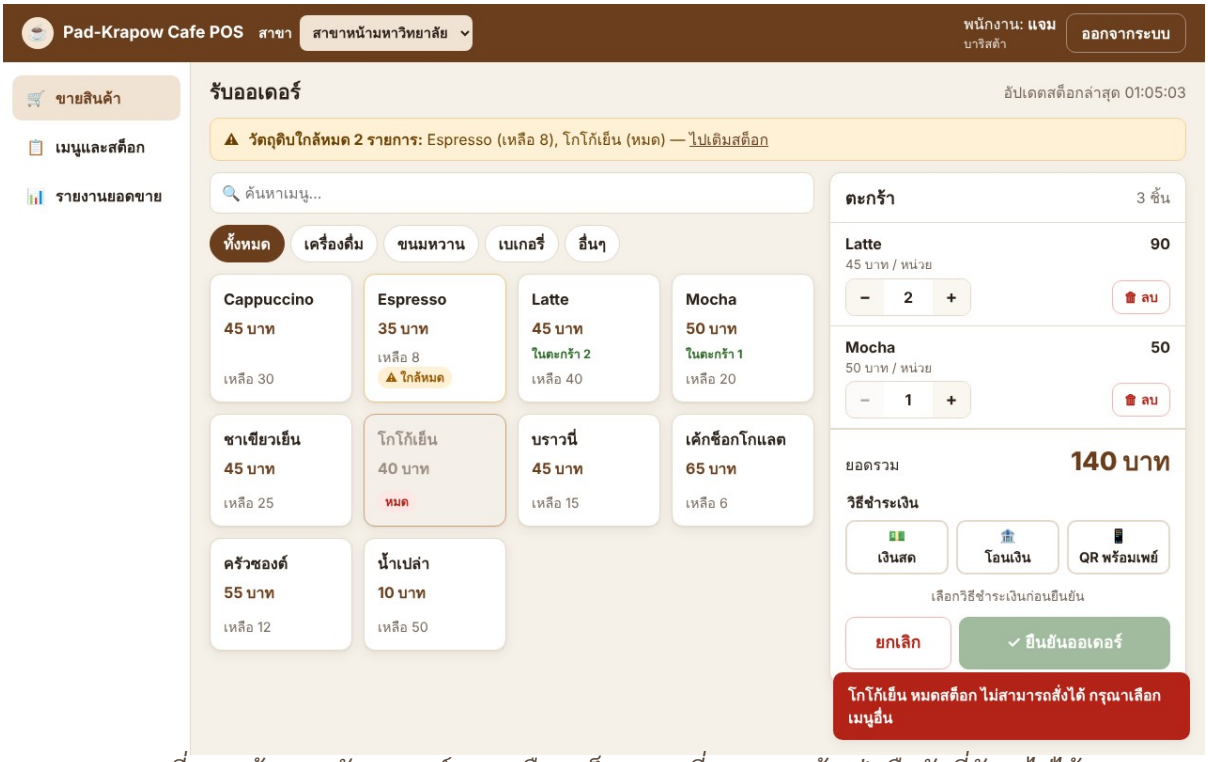
การอัปเดตสต็อกข้ามเครื่อง

หน้า POS ดึงข้อมูลเมนูและสต็อกใหม่ทุก 3 วินาทีเมื่อเปิดหน้าอยู่ ทดสอบด้วย 2 เครื่อง เครื่องที่สองเห็นสต็อกใหม่ภายใน 2.8 วินาที (NFR-04)

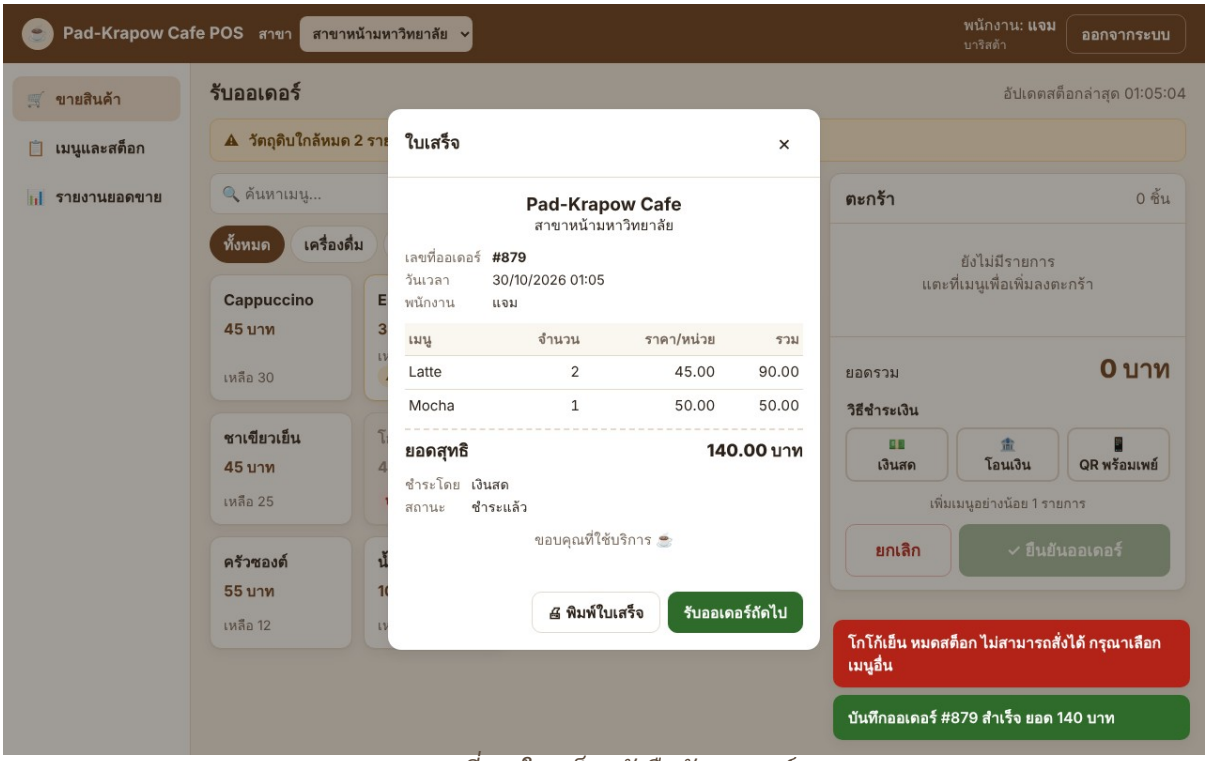
5.4 REST API

Method	Endpoint	หน้าที่
POST	/api/auth/login	เข้าสู่ระบบ
GET	/api/branches	รายชื่อสาขาที่เข้าถึงได้
GET	/api/menu, /api/menu/low-stock, /api/menu/categories, /api/menu/:id	เมนู สต็อก หมวดหมู่
POST / PATCH / DELETE	/api/menu, /api/menu/:id	จัดการเมนู (เจ้าของร้าน)
PATCH	/api/menu/:id/stock	เติม/ปรับสต็อก
POST	/api/orders	สร้างออเดอร์ + ตัดสต็อก
GET	/api/orders?date=, /api/orders/:id	ออเดอร์ของวัน / ใบเสร็จ
GET	/api/reports/sales, /api/reports/monthly	รายงานยอดขาย

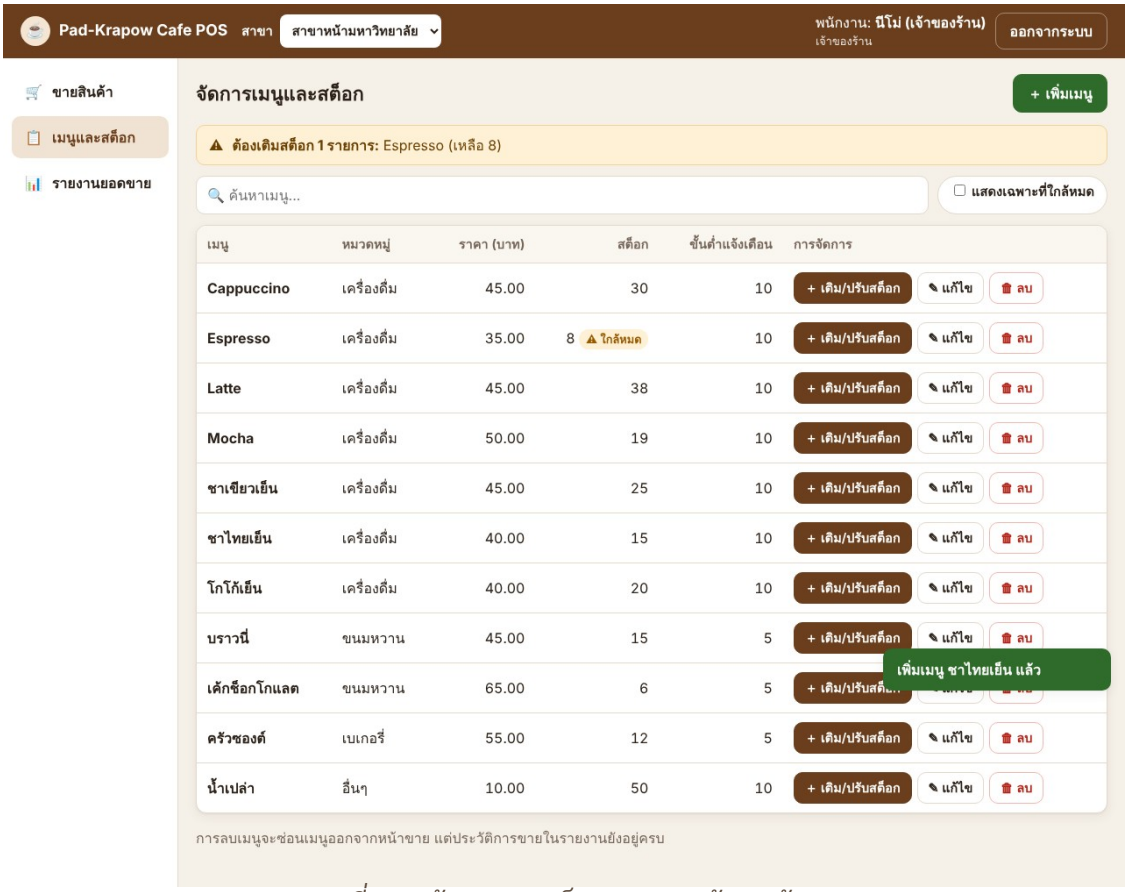
5.5 หน้าจอระบบ



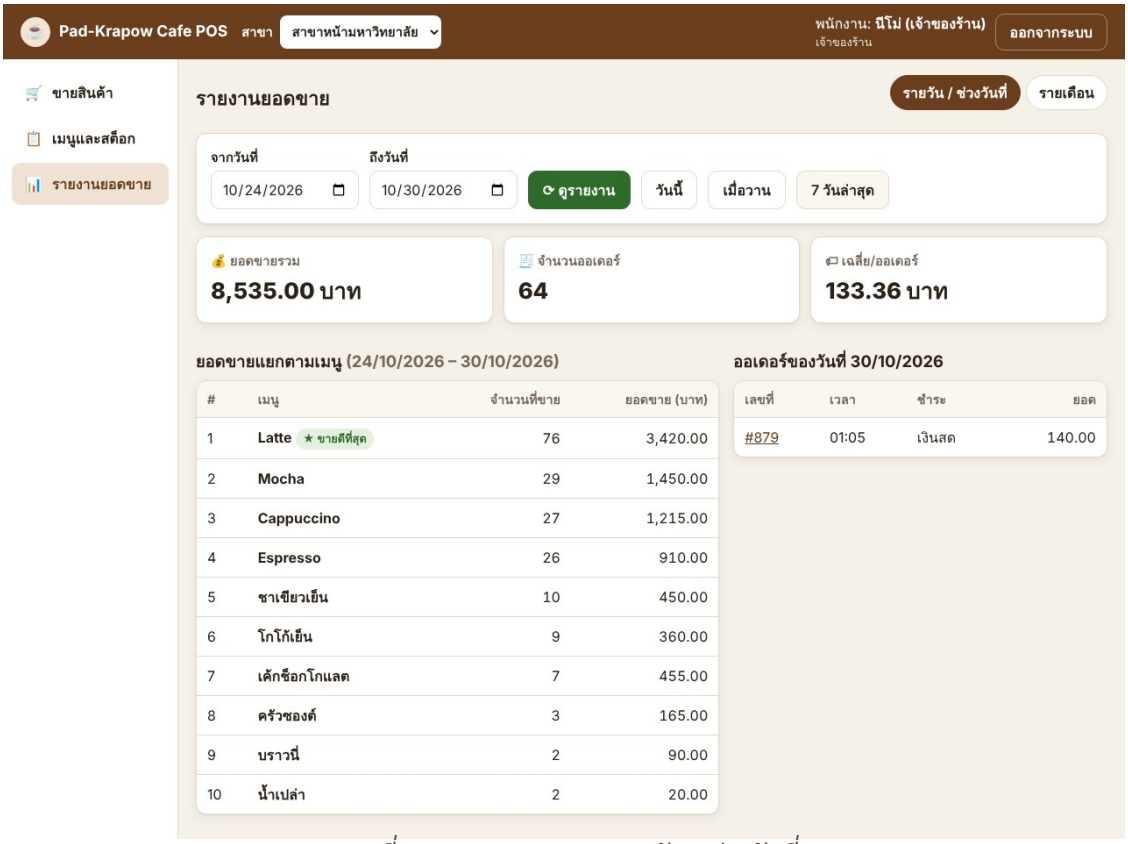
ภาพที่ 11 หน้า POS รับออเดอร์ (แถบเตือนสต็อก, เมนูที่หมด, ตะกร้า, ปุ่มยืนยันที่ยังกดไม่ได้)



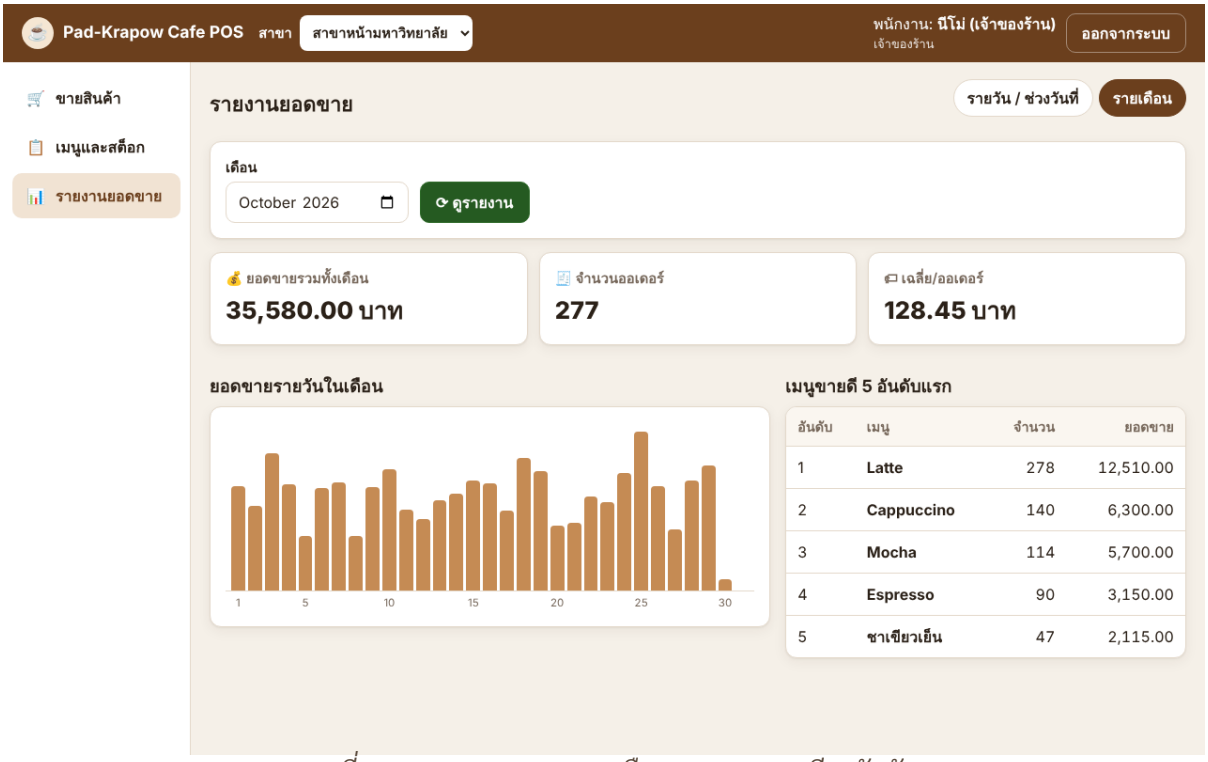
ภาพที่ 12 ใบเสร็จหลังยืนยันออเดอร์



ภาพที่ 13 หน้าเมนูและสต็อก (มุมมองเจ้าของร้าน)



ภาพที่ 14 รายงานยอดขายรายวัน / ช่วงวันที่



ภาพที่ 15 รายงานยอดขายรายเดือน และเมนูขายดี 5 อันดับ



ภาพที่ 16 หน้า POS บนจอกว้าง 390px

บทที่ 6 การทดสอบระบบ

6.1 การทดสอบอัตโนมัติ

ชุดทดสอบ `npm test` มี 41 กรณี (unit 8 + API 33) ทดสอบกับฐานข้อมูลจริง ผูกกับ Acceptance Criteria ของ User Story ผลทดสอบ **ผ่าน 41/41** บน MySQL 8.0.46 และ MariaDB 10.11

ตารางที่ 6 สรุปกลุ่ม Test Case

กลุ่ม	Test Case	ตัวอย่างที่ทดสอบ	ผล
การเข้าสู่ระบบและสิทธิ์	TC-01 – TC-08	bcrypt, 401/403/429, กัน IDOR ข้ามสาขา	8/8 ผ่าน
สร้างออเดอร์	TC-10 – TC-21	ยอดรวม, วิธีชำระ, รวมเมนูซ้ำ, ไม่เชื่อราคา client, สต็อกไม่พอ rollback, ขายพร้อมกัน	12/12 ผ่าน
เมนูและสต็อก	TC-30 – TC-35	สิทธิ์เจ้าของร้าน, validation, เติมสต็อก, soft delete	6/6 ผ่าน
รายงาน	TC-40 – TC-45	ยอดรายวัน, ขอบเวลา 23:59/00:00, รายเดือน, Top 5, วันที่ไม่ถูกต้อง	6/6 ผ่าน
ความปลอดภัยทั่วไป	TC-50	404 JSON, security headers	1/1 ผ่าน
Unit	8 กรณี	Order/OrderItem, ฟังก์ชันตรวจข้อมูล	8/8 ผ่าน

6.2 การทดสอบ End-to-End ผ่านหน้าจอ

ID	สิ่งที่ทดสอบ	ผลจริง
E2E-01	login รหัสผิด	แสดง "ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง"
E2E-02	Latte×2 + Mocha×1	ยอดรวม 140 บาท
E2E-03	แตะเมนูที่หมด	ไม่เพิ่มลงตะกร้า แจ้ง "หมดสต็อก"
E2E-04	ยังไม่เลือกวิธีชำระ	ปุ่มยืนยัน disabled + บอกเหตุผล
E2E-05	ยืนยันออเดอร์	แสดงใบเสร็จ ยอด 140.00 ชำระโดยเงินสด
E2E-06	บาริสต้าเติมสต็อก	สต็อกอัปเดต ไม่มีปุ่มแก้ไข/ลบ
E2E-07	2 เครื่องพร้อมกัน	อีกเครื่องเห็นสต็อกใหม่ใน 2.8 วินาที
E2E-08	จอกว้าง 390px	ไม่มี scroll แนวนอน
E2E-09	เจ้าของร้านเพิ่มเมนู	เมนูใหม่ขึ้นในตาราง
E2E-10/11	รายงานรายวัน/รายเดือน/เปลี่ยนสาขา	แสดงข้อมูลถูกต้องตามสาขา

6.3 การทดสอบประสิทธิภาพ (NFR-01)

วัดเวลาบันทึกออเดอร์ผ่าน HTTP 10 ครั้ง (ฐานข้อมูลมีออเดอร์ประมาณ 900 รายการ) ค่ากลาง 8 ms ช้าสุด 22 ms ต่ำกว่าเกณฑ์ 2 วินาทีมาก ควรวัดซ้ำบนเครื่องจริงของร้าน

6.4 Usability Testing

ทดสอบแบบ Think-Aloud 3 Task (wk12) พบปัญหาระดับ Minor 2 ข้อ และแก้ใน UI แล้ว

Issue	ปัญหา	การแก้
#1	หาปุ่มยืนยันออเดอร์ช้า	ปุ่มใหญ่สีเขียวล่างตะกร้า + ข้อความบอกสิ่งที่ยังขาด
#2	หาทางเข้าหน้ารายงานไม่เจอ	เมนูด้านซ้ายแสดงทุกหน้า เข้าได้ใน 1 คลิก + ป้าย "ขายดีที่สุด"

ข้อจำกัด: บันทึก Think-Aloud ใน wk12 ยังเป็นข้อมูลตัวอย่าง ต้องทดสอบซ้ำกับผู้ใช้จริงบนระบบที่พัฒนาแล้ว (มีแผนและตารางบันทึกผลใน wk12 หัวข้อ 5)

บทที่ 7 การตรวจสอบคุณภาพ

ตรวจทั้ง 16 endpoint ตาม checklist (wk14)

หัวข้อ	ผล
Input Validation	ผ่าน — ตรวจชนิด/ช่วงค่าทุก field ที่ server
Authentication	ผ่าน — JWT + bcrypt + จำกัดการเดารหัสผ่าน
สิทธิ์ / IDOR	ผ่าน — แยกสาขาและบทบาท
SQL Injection	ผ่าน — parameterized query ทุกจุด
XSS	ผ่าน — escape ทุกจุด + Content-Security-Policy
Error handling	ผ่าน — JSON ภาษาไทย ไม่เปิดเผย stack trace
Race condition สต็อก	ผ่าน — transaction + FOR UPDATE + CHECK

7.1 สิ่งที่แก้จากโค้ดเดิม

- ตัดสต็อกโดยไม่ตรวจว่าพอ → ตรวจหลังล็อกแถว และ rollback ถ้าไม่พอ
- รับราคาและรหัสพนักงานจาก client → ใช้ราคาจากฐานข้อมูล และรหัสพนักงานจาก token
- ไม่มี login / ไม่ตรวจสิทธิ์สาขา → เพิ่ม login และ middleware ตรวจสิทธิ์
- ไม่ใช้ตาราง stock_movement → บันทึกทุกครั้งที่สต็อกเปลี่ยน

7.2 ประเด็นที่ยังควรปรับปรุง

- ยังไม่มี HTTPS (เหมาะใช้ใน WiFi ของร้านเท่านั้น)
- token เก็บใน localStorage และยังใช้ได้จนหมดอายุหลังออกจากระบบ
- ยังไม่บันทึกว่าใครเป็นผู้ปรับสต็อก และยังไม่มีการยกเลิกออเดอร์

บทที่ 8 สรุปและข้อเสนอแนะ

8.1 สรุปผล

ระบบ Pad-Krapow-cafe-system ทำงานได้ตาม Functional Requirement FR-01 ถึง FR-13 แก้ปัญหาทั้ง 3 ข้อของร้าน คือ คิดเงินผิด (ระบบคำนวณให้และตรวจซ้ำที่ server), สต็อกหมดโดยไม่รู้ตัว (แสดงสต็อก แจ้งเตือน และห้ามขายเกิน) และไม่มีข้อมูลยอดขาย (รายงานรายวัน/รายเดือน และเมนูขายดี) ผ่านการทดสอบอัตโนมัติ 41/41 และ End-to-End 11 กรณี

8.2 ข้อจำกัด

- ยังไม่รองรับการทำงานเมื่ออินเทอร์เน็ต/WiFi หลุด
- สต็อกนับเป็นจำนวนต่อเมนู ยังไม่แยกเป็นวัตถุดิบ
- ยังไม่ได้ทดสอบกับผู้ใช้จริงบนแท็บเล็ตของร้าน

8.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ

1. เพิ่มสูตรวัตถุดิบต่อเมนู (เช่น Latte 1 แก้ว = นม 200 ml + กาแฟ 18 g) เพื่อตัดสต็อกวัตถุดิบจริง
2. เพิ่มการยกเลิก/คืนเงินออเดอร์ และบันทึกผู้ปรับสต็อก
3. ติดตั้ง HTTPS และเปลี่ยนไปเก็บ token ใน HttpOnly cookie
4. ใช้ WebSocket แทนการดึงข้อมูลทุก 3 วินาทีเมื่อมีเครื่องมากขึ้น
5. สร้าง QR พร้อมเพย์ตามยอดเงินบนหน้าจอให้ลูกค้าสแกนได้ทันที

ภาคผนวก

ก. วิธีติดตั้งและรันระบบ

1. ติดตั้ง Node.js 20+ และ MySQL 8.0 หรือ MariaDB (เช่น XAMPP)
2. `cd code` แล้ว `npm install`
3. คัดลอก `.env.example` เป็น `.env` แล้วแก้ DB_USER / DB_PASSWORD
4. `npm run db:setup` (หรือ `npm run db:setup:demo` เพื่อเพิ่มออเดอร์ตัวอย่าง)
5. `npm start` แล้วเปิด http://localhost:3000
6. ทดสอบด้วย `npm test`

ข. บัญชีสำหรับทดลอง

ชื่อผู้ใช้	รหัสผ่าน	บทบาท
owner	Owner@1234	เจ้าของร้าน
barista1	Barista@1234	บาริสต้า สาขาหน้ามหาวิทยาลัย
barista2	Barista@1234	บาริสต้า สาขาตัวเมือง

ค. การใช้ AI ในโครงงาน

กลุ่มใช้ AI ช่วยงานในหลายสัปดาห์ รายละเอียดเครื่องมือ ส่วนที่ใช้ และสมาชิกที่รับผิดชอบอยู่ในโฟลเดอร์ `docs/ai-disclosure/` (wk05, wk09, wk10, wk13) ในสัปดาห์ที่ 13 ใช้ Claude (Anthropic) ช่วยเขียนโค้ดระบบ ชุดทดสอบ และเอกสารส่วนที่ขาด โดยอ้างอิงเอกสารออกแบบของกลุ่ม สมาชิกกลุ่มเป็นผู้ตรวจสอบ ทดสอบ และรับผิดชอบความถูกต้องของงาน

ง. เอกสารอ้างอิงใน Repository

สัปดาห์	เอกสาร
1–2	wk01-team-charter.md, wk02-feasibility-report.md
3–4	wk03-requirements.md, wk04-user-stories.md, use-case-diagram.png
6–7	wk06-class-diagram.png, wk06-object-diagram.png, wk07-er-diagram.png, wk07-schema.sql
9–10	wk09-sequence-diagram.png, wk10-srs.md, wk10-wireframe.docx, wk10-input-output-design.docx
11–12	wk11-architecture.md, wk12-usability-testing.md
13–14	wk13-test-cases.md, wk14-quality-review.md